

Grundlagen "Web-Engineering"

World Wide Web



"HyperText is a way to link and access information of various kinds as a web of nodes in which the user can browse at will. Potentially, HyperText provides a single user-interface to many large classes of stored information such as reports, notes, data-bases, computer documentation and on-line systems help. We propose the implementation of a simple scheme to incorporate several different servers of machine-stored information already available at CERN, including an analysis of the requirements for information access needs by experiments."

(<https://www.w3.org/Proposal.html>)

Das **World Wide Web** (englisch für „weltweites Netz“, kurz Web oder WWW) ist ein über das **Internet** abrufbares System von [...] sogenannten **Webseiten**, welche mit **HTML** beschrieben werden.

Sie sind durch **Hyperlinks** untereinander verknüpft und werden im Internet über die Protokolle **HTTP** oder **HTTPS** übertragen.

Zum Aufrufen von Inhalten aus dem World Wide Web wird ein **Webbrowser** benötigt, der z. B. auf einem **PC** oder einem **Smartphone** läuft. Mit ihm kann der Benutzer die auf einem beliebigen, von ihm ausgewählten **Webserver** bereitgestellten Daten herunterladen und auf einem geeigneten Ausgabegerät wie einem **Bildschirm** oder einer **Braillezeile** anzeigen lassen.

(vgl. https://de.wikipedia.org/wiki/World_Wide_Web)

Engineering

Engineering is the practice of using **natural science, mathematics, and the engineering design** process to solve **technical problems**, increase **efficiency** and **productivity**, and **improve systems**.

The **engineering design process** is a common series of steps that engineers use in creating **functional products and processes**. The process is **highly iterative**[...]

It is a **decision making process** in which the **basic sciences** [...] are **applied** to convert resources optimally **to meet a stated objective**.

Among the fundamental elements of the design process are the **establishment of objectives and criteria, synthesis, analysis, construction, testing and evaluation**.

(vgl. <https://en.wikipedia.org/wiki/Engineering>,
https://en.wikipedia.org/wiki/Engineering_design_process)

- **Anwendung von Wissenschaften**
- Lösen von technischen Problemen, erhöhen der Effizienz, Optimierung von Systeme
- **Iterativer Prozess**
- Entscheidungsverfahren
- Bestimmen der Anforderungen und **Analyse, Herstellung, Überprüfung und Evaluation**

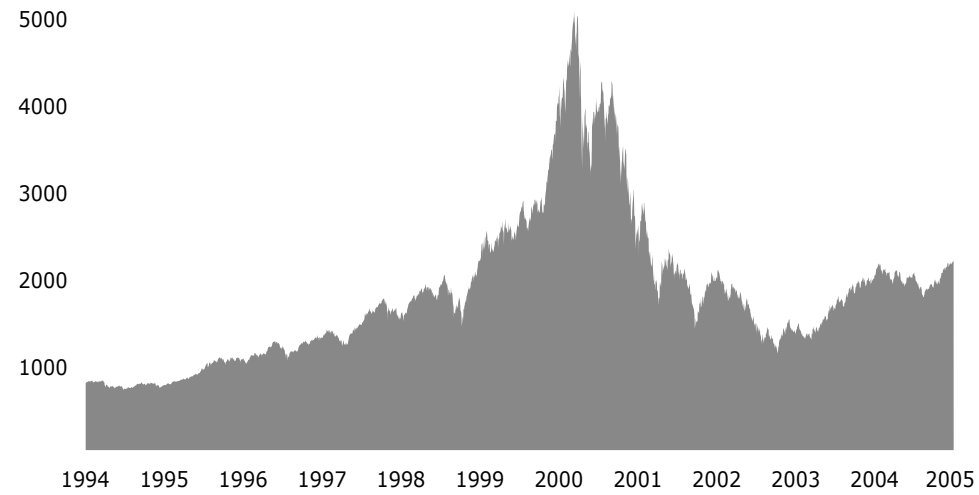
History of the Web

Web 1.0

- WWW (1989): Tim Berners-Lee, CERN:
<https://info.cern.ch/hypertext/WWW/TheProject.html>
- HTTP (1991): <https://datatracker.ietf.org/doc/html/rfc1945>
- W3C (1994): <https://www.w3.org/>

(vgl. https://en.wikipedia.org/wiki/History_of_the_World_Wide_Web)

dot-com bubble



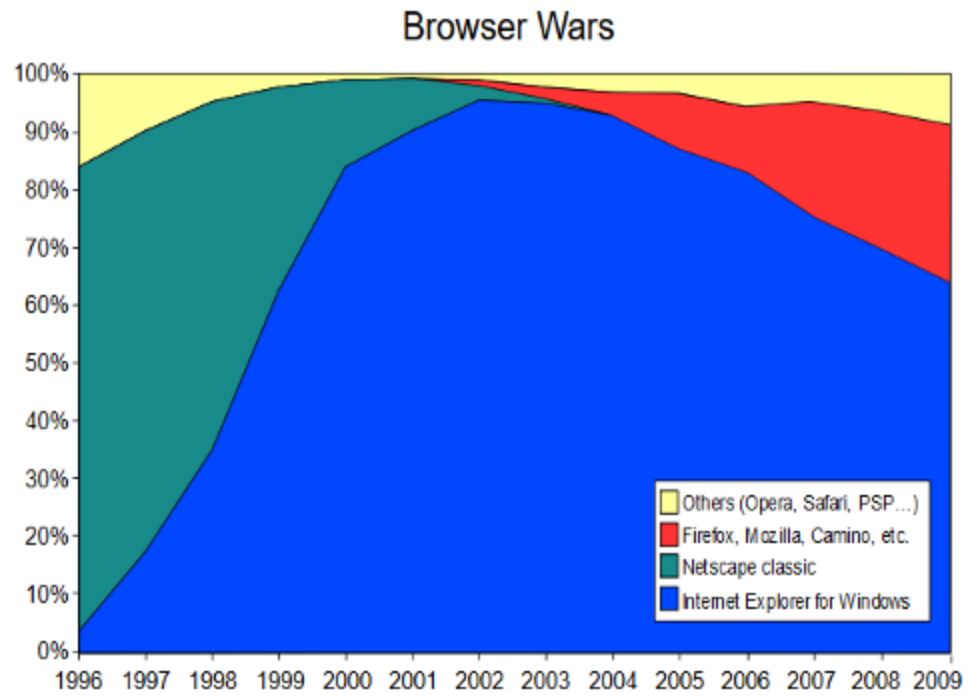
Nasdaq Composite index

https://en.wikipedia.org/wiki/Dot-com_bubble

Web 2.0

- Social Media: Youtube (2005), Facebook (2004), MySpace (2003), Twitter (2006)
- JavaScript (1995): Brendan Eich, Netscape
- Ajax (2000): XMLHttpRequest
- Web Applications, SPA, PWA: Gmail (2004), Google Maps (2005)
- Cloud Computing: AWS (2002)
- Mobile: iPhone (2007)

Browser Wars



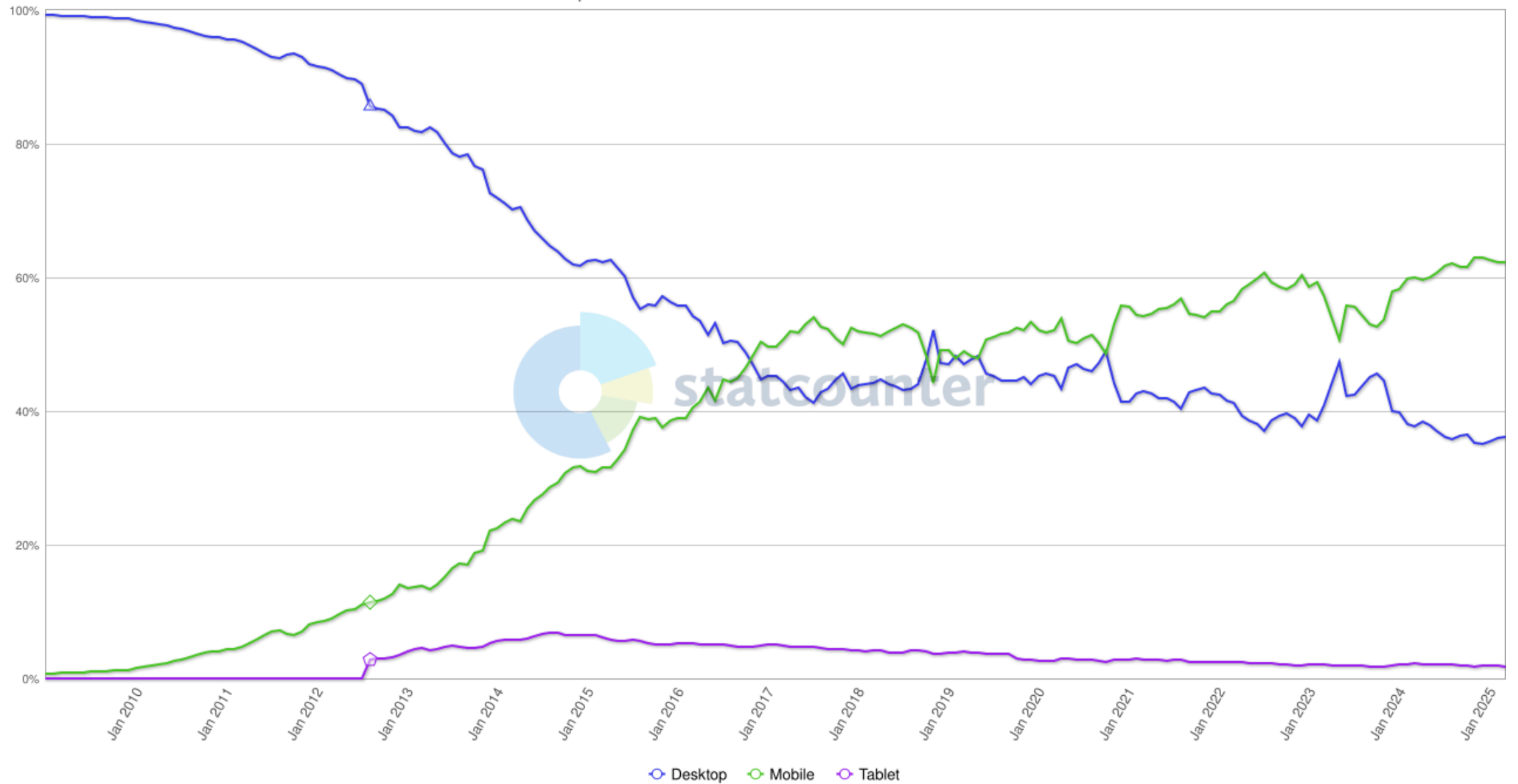
StatCounter Global Stats

Browser Market Share Worldwide from Jan 2009 - Mar 2025



statcounter

StatCounter Global Stats
Desktop vs Mobile vs Tablet Market Share Worldwide from Jan 2009 - Mar 2025



Web 3.0 / Semantic Web

- Tim Berners-Lee, 1999
- Daten werden mit Metadaten maschinenlesbar aufbereitet
- Dadurch können Informationen verlinkt werden
- Beispiel: RSS, <https://hnrss.org/frontpage>

Web3

- Dezentrales Web
- Blockchains
- smart contracts
- digital tokens

Gegenwart und Zukunft

- Browser als App-Plattform
- Local First: <https://localfirstweb.dev/>
- Internet of Things?
- "Dead Internet Theory" / Enshittification / AI Slop
 - <https://www.wired.com/story/tiktok-platforms-cory-doctorow/>
 - <https://www.zeit.de/digital/internet/2025-04/plattformverfall-enshittification-cory-doctorow-facebook-internet>

Werkzeuge

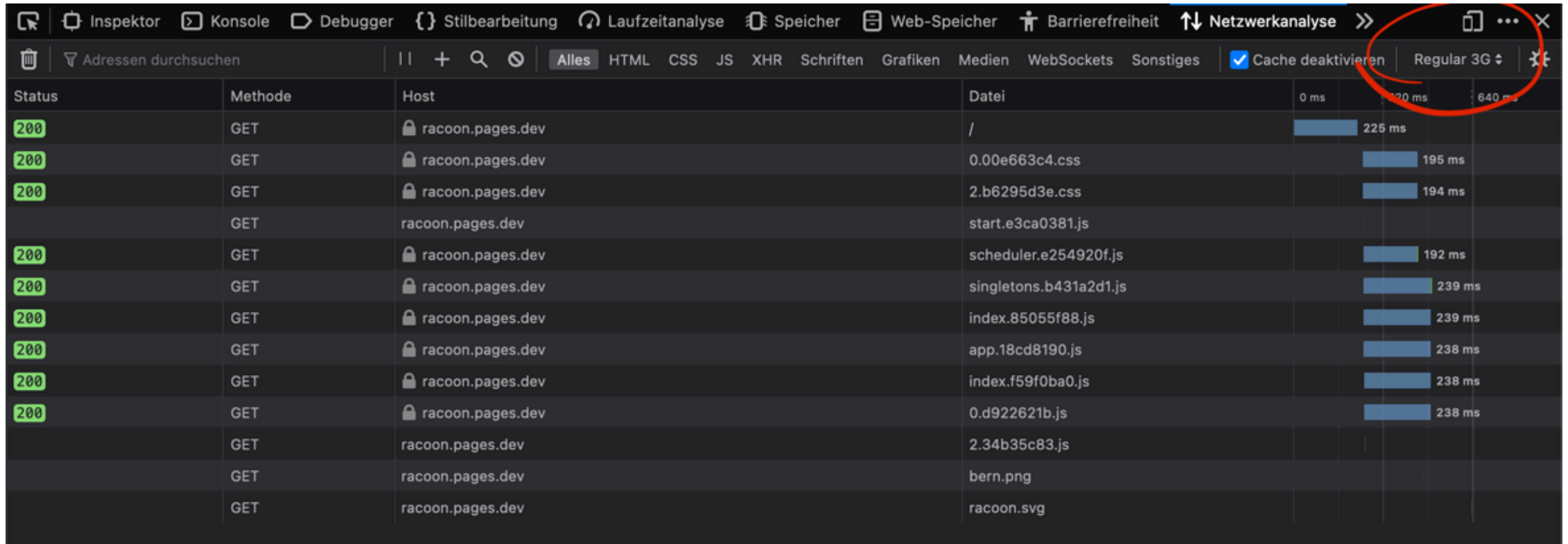
- <https://developer.mozilla.org/>
- <https://www.mozilla.org/de/firefox/windows/>
- <https://www.google.com/intl/de/chrome/>
- <https://www.jetbrains.com/webstorm/>
- Git: <https://git-scm.com/>
- Node.js: <https://nodejs.org/en>
- Docker: <https://www.docker.com/>
- <https://caniuse.com/>

Tools.md

Browser Tools: Inspektor

The image is a composite of two parts. On the left, a dark-themed web form for a newsletter subscription is shown. It includes a heading 'Dispatch Software' with a subtext 'to enable bike messenger companies to focus on their core business: Deliver goods fast, simple, reliable, flexible and environmental friendly.' Below this is a form with an 'Email' input field, a checkbox labeled 'Apply for early access', and a 'Subscribe' button. A small status bar at the bottom of the form shows 'h2#fast' and '866.4 x 44'. On the right, the source code of the form is displayed in a light-themed editor. The code is a mix of HTML and CSS. The HTML part shows the form's structure, including a 'form' element with a 'class' attribute and a 'data' attribute. The CSS part is a large color palette with various shades of blue, gray, and green, along with a dark theme scheme. The palette includes a list of colors with their corresponding hex codes and a section for a dark theme scheme with various shades of gray and blue.

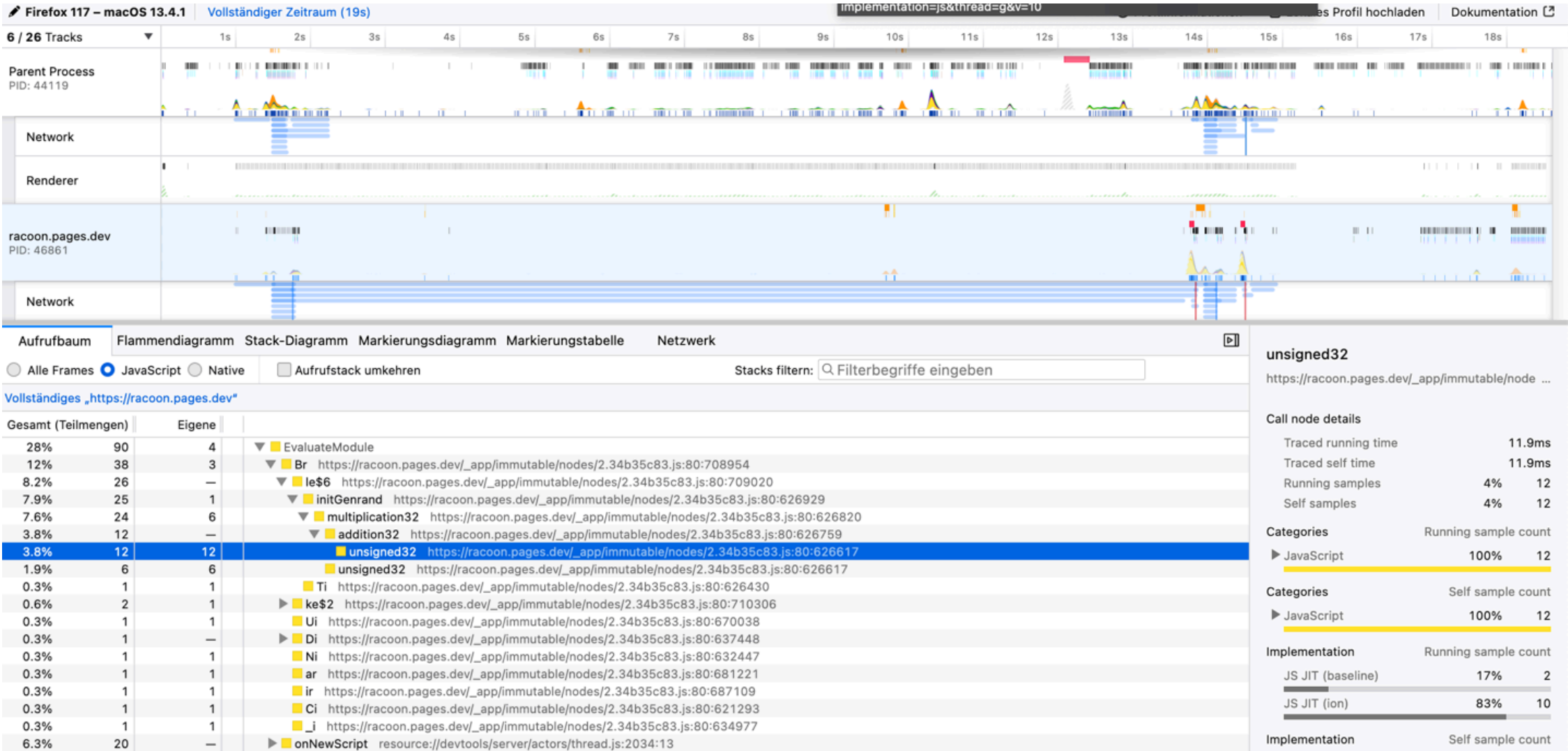
Browser Tools: Netzwerkanalyse



The screenshot shows the Chrome DevTools Network tab. The top toolbar includes various tool icons and the 'Netzwerkanalyse' (Network) tab is selected. Below the toolbar, there's a search bar and a list of network requests. The 'Cache deaktivieren' checkbox is checked, and the 'Regular 3G' throttling dropdown is visible. A red circle highlights these two elements.

Status	Methode	Host	Datei	0 ms	20 ms	640 ms
200	GET	racoon.pages.dev	/	225 ms		
200	GET	racoon.pages.dev	0.00e663c4.css	195 ms		
200	GET	racoon.pages.dev	2.b6295d3e.css	194 ms		
	GET	racoon.pages.dev	start.e3ca0381.js			
200	GET	racoon.pages.dev	scheduler.e254920f.js	192 ms		
200	GET	racoon.pages.dev	singletons.b431a2d1.js	239 ms		
200	GET	racoon.pages.dev	index.85055f88.js	239 ms		
200	GET	racoon.pages.dev	app.18cd8190.js	238 ms		
200	GET	racoon.pages.dev	index.f59f0ba0.js	238 ms		
200	GET	racoon.pages.dev	0.d922621b.js	238 ms		
	GET	racoon.pages.dev	2.34b35c83.js			
	GET	racoon.pages.dev	bern.png			
	GET	racoon.pages.dev	racoon.svg			

Browser Tools: Laufzeitanalyse



Webseiten strukturieren mit HTML

Sprache definieren

```
<html lang="en-US">  
  ...  
</html>
```

Meta Daten

```
<meta name="author" content="Chris Mills" />
<meta
  name="description"
  content="The MDN Web Docs site
  provides information about Open Web technologies
  including HTML, CSS, and APIs for both websites and
  progressive web apps." />

<meta charset="utf-8" />
<meta name="color-scheme" content="dark light" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

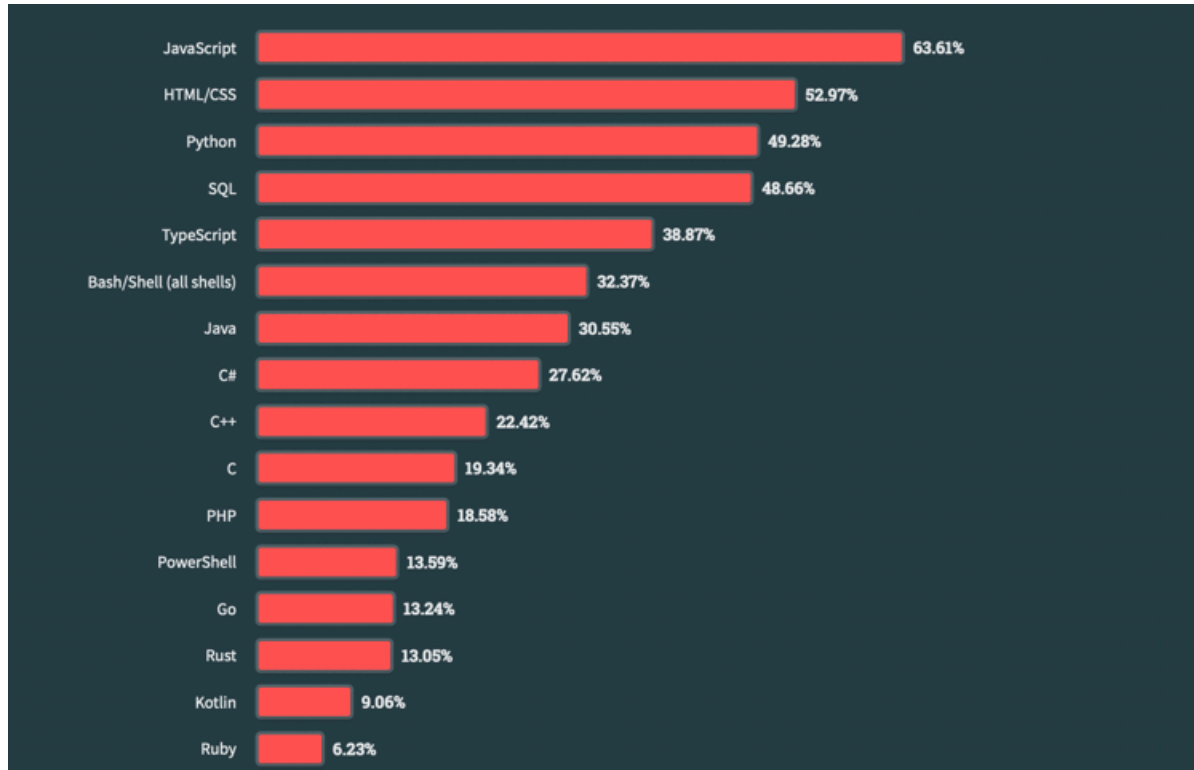
Icons

```
<link rel="icon" href="/favicon.ico" />  
<link rel="manifest" href="manifest.json" />
```

```
// manifest.json  
{  
  "icons": [  
    {  
      "src": "basic-icon.png"  
    }  
  ]  
}
```

State of JavaScript

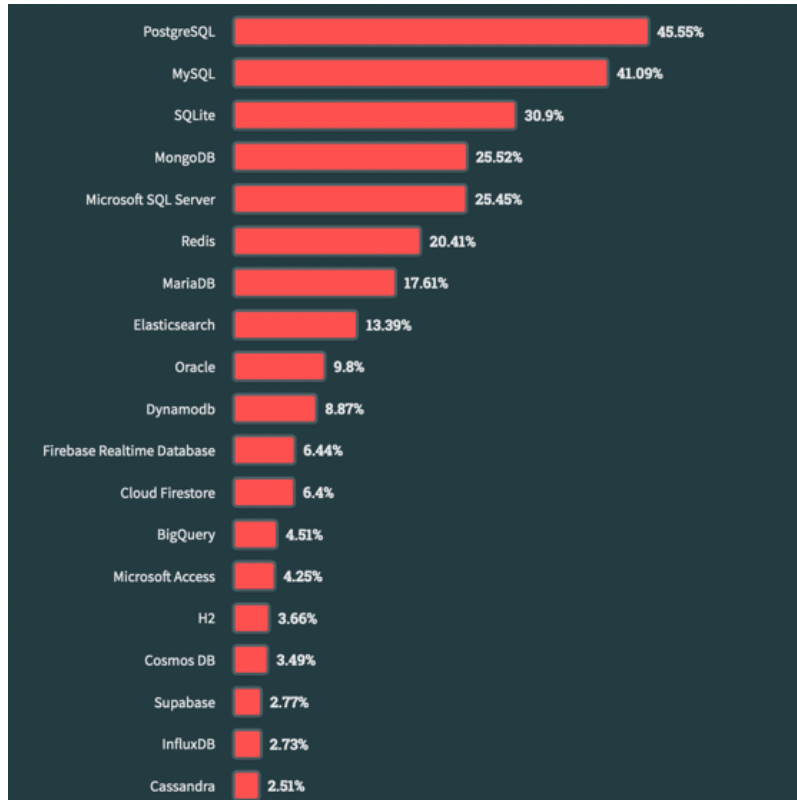
Programmiersprachen



<https://survey.stackoverflow.co/2023/#section-most-popular-technologies-programming-scripting-and-markup-languages>,

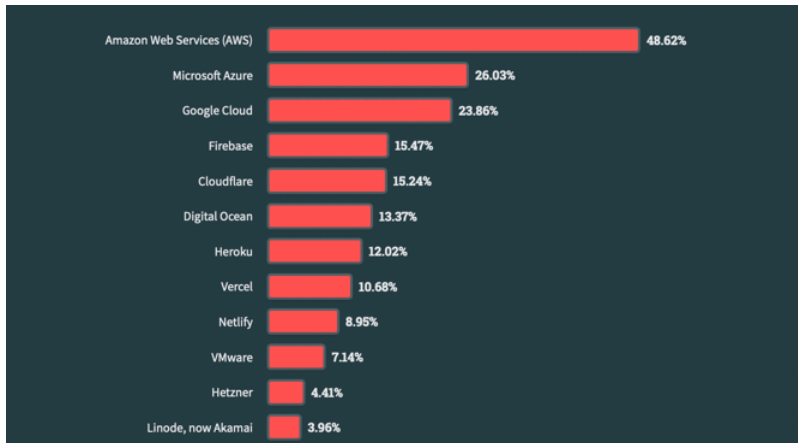
23.04.24

Datenbanken



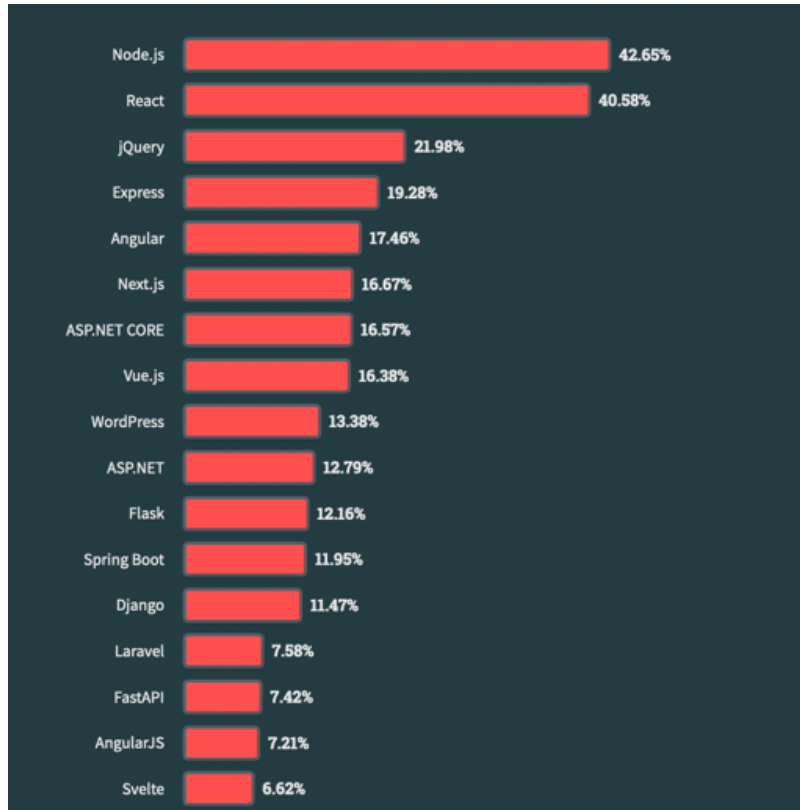
<https://survey.stackoverflow.co/2023/#section-most-popular-technologies-databases>,
23.04.24

Cloud Plattformen



<https://survey.stackoverflow.co/2023/#section-most-popular-technologies-cloud-platforms>, 23.04.24

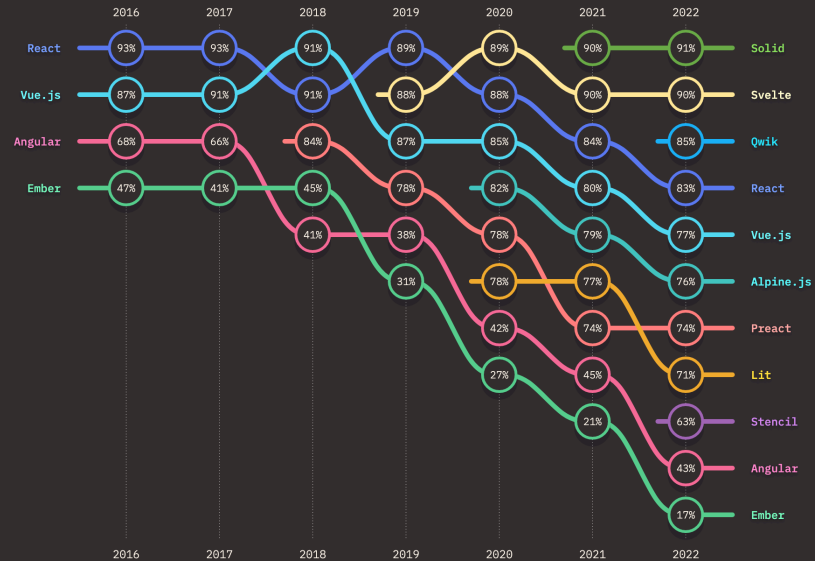
Web Frameworks



<https://survey.stackoverflow.co/2023/#section-most-popular-technologies-web-frameworks-and-technologies>, 23.04.24

RANKINGS OVER TIME

Retention, interest, usage, and awareness ratio rankings.



Retention Interest Usage Awareness

Technologies with less than 10% awareness not included. Each ratio is defined as follows:

- Retention: $\frac{\text{would use again}}{\text{would use again} + \text{would not use again}}$
- Interest: $\frac{\text{want to learn}}{\text{want to learn} + \text{not interested}}$
- Usage: $\frac{\text{would use again} + \text{would not use again}}{\text{total}}$
- Awareness: $\frac{\text{total} - \text{never heard}}{\text{total}}$

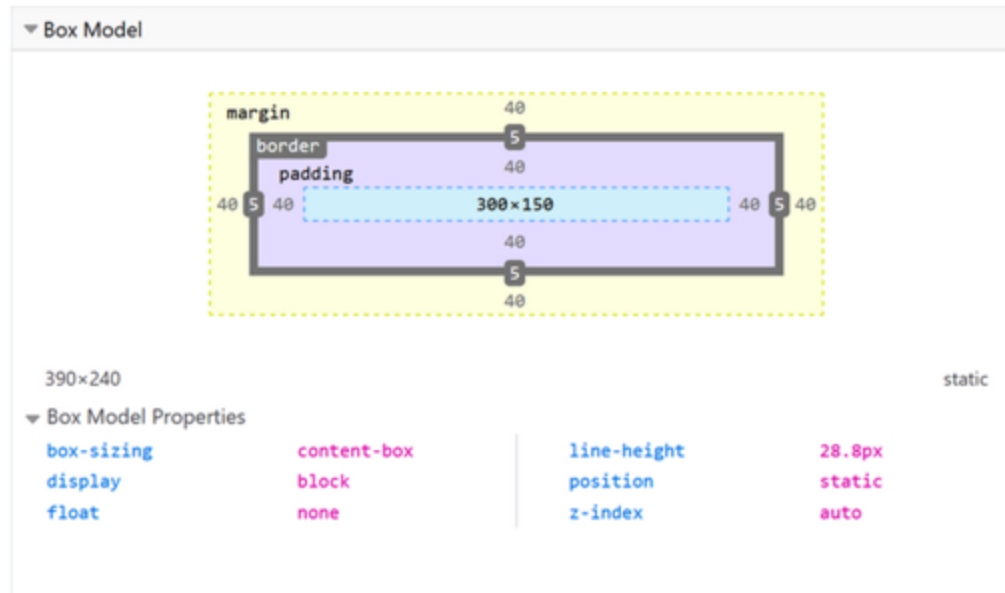
State of JavaScript 2022, 21.06.2023

Webseiten gestalten mit CSS

Layoutkonzepte

- <http://info.cern.ch/hypertext/WWW/TheProject.html>
- Framesets
- Tabellen
- Cascading Style Sheets (CSS)
- Fixed vs. Liquid Layout
- Responsive Webdesign
- Device Agnostic
- Mobile First

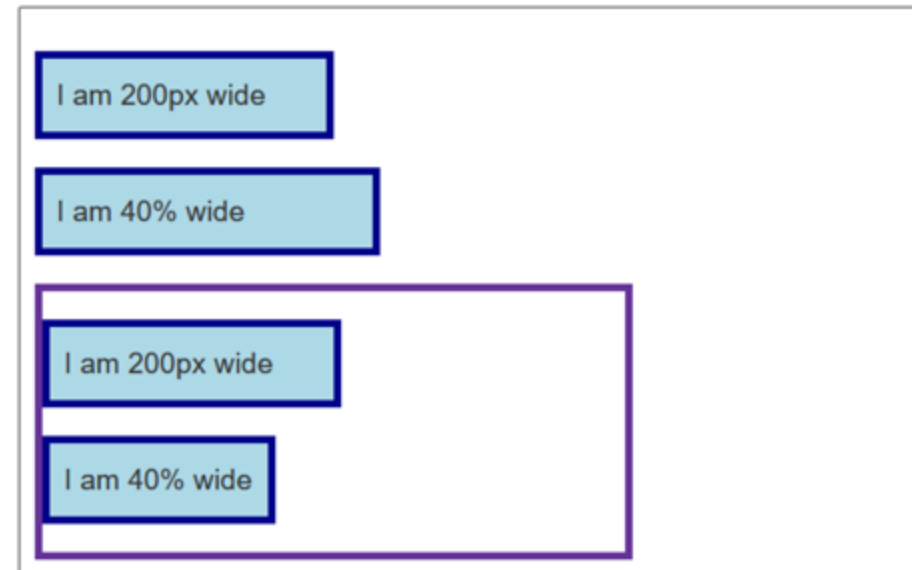
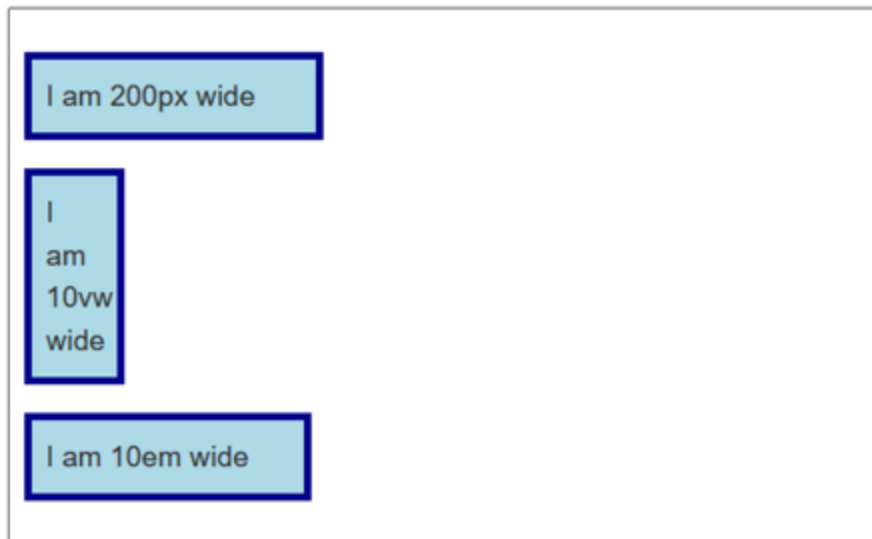
Box Model



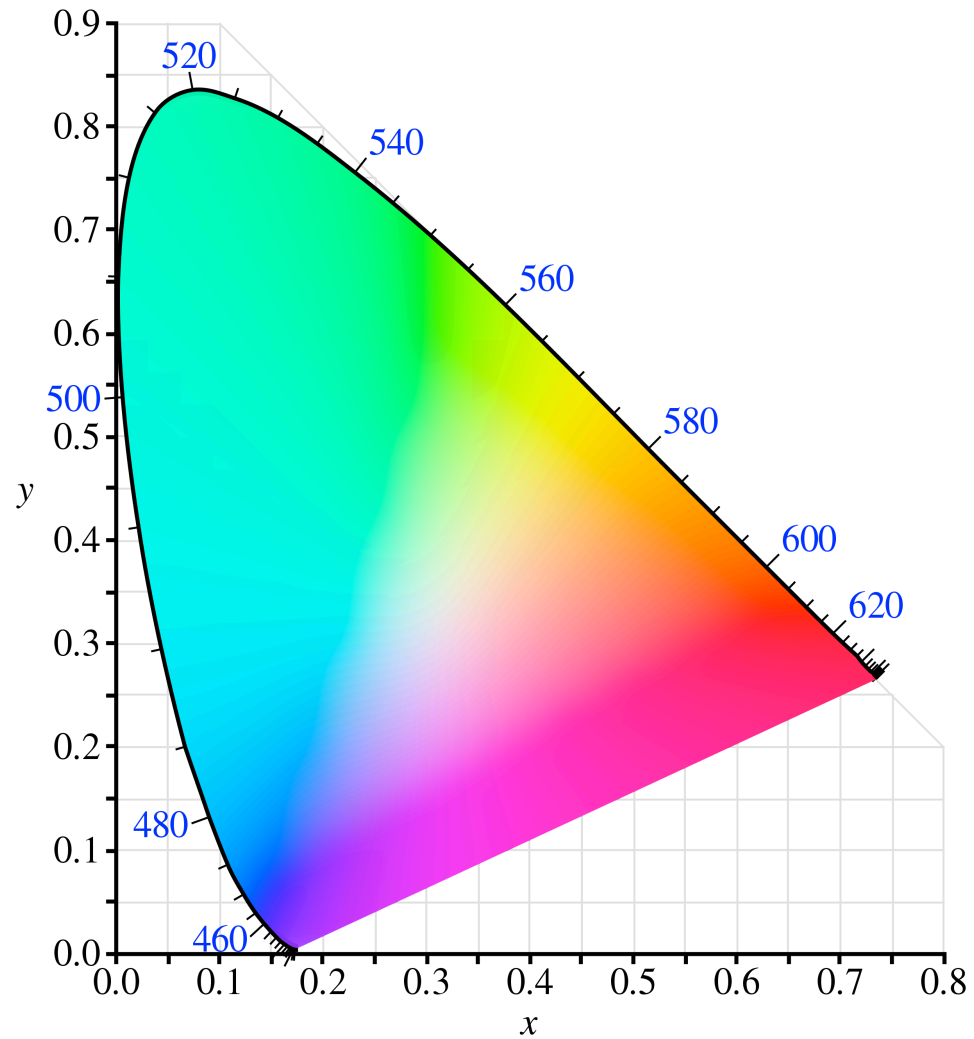
- `box-sizing: content-box: width` bezieht sich nur auf den content (blau, 300x150)
- `box-sizing: border-box: width` bezieht sich auf content + padding + border (blau, violett, grau, $300+2*40+2*5$ für die Breite)

Einheiten

- Absolute Grössen: `px` (`cm` , `mm` , ...) -> sparsam verwenden
- Relative Grössen
 - `em` : Schriftgrösse des Elternelements
 - `rem` : Schriftgrösse des Wurzelements
 - `vw` , `vh` : viewport breite, viewport höhe

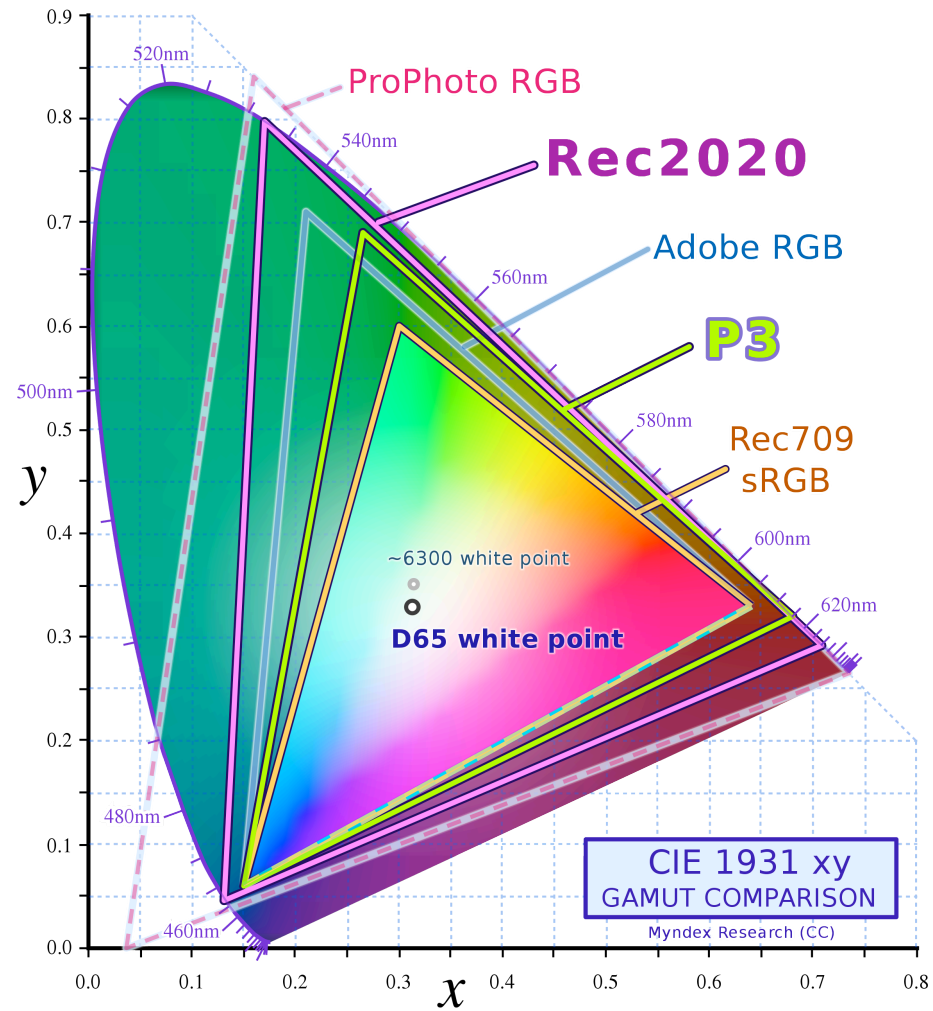


Farben



CIE 1931 Farbraum

Vergleich Farbräume



Farben in CSS

sRGB Farbraum

- Farbnamen: `color: darkblue;`
- Hex-Werte: `color: #ffa500;`
- RGB-Werte (Rot, Grün, Blau, Deckkraft): `color: background: background: rgb(30% 20% 50% / 0.7);`
- HSL-Werte (Farbe, Sättigung, Helligkeit, Deckkraft): `color: hsl(150deg 80% 40% / 0.7);`

Alle sichtbaren Farben

- LCH (Lightness Chroma Hue / Opacity): `color: lch(29% 44 27 / 0.5)`
- Oklch: `color: oklch(40% 0.1 21)`
- CIELAB (Lightness, red-green, blue-yellow): `color: lab(29% 39 20);`
- Oklab: `color: oklab(40. 0.1 0.04);`

HSL vs LCH

HSL	50%	54%	50%	50%	LCH
	50%	68%	50%	50%	
	50%	97%	50%	50%	
	50%	90%	50%	50%	
	50%	89%	50%	50%	
	50%	88%	50%	50%	
	50%	91%	50%	50%	
	50%	53%	50%	50%	
	50%	30%	50%	50%	
	50%	39%	50%	50%	
	50%	60%	50%	50%	
	50%	56%	50%	50%	

50% Luminance in HSL (links) vs in LCH (rechts)

<https://codepen.io/web-dot-dev/pen/poZgXxy>

Farbsysteme

```
:root {
  color-scheme: light dark;

  --color-neutral-25: hsl(0deg 0% 99%);
  --color-neutral-50: hsl(0deg 0% 97%);
  /* ... */
  --color-neutral-900: hsl(0deg 0% 8%);
  --color-neutral-950: hsl(0deg 0% 4%);

  --color-blue-100: oklch(0.95 0.17 264);
  --color-blue-200: oklch(0.90 0.17 264);
  /* ... */
  --color-blue-500: oklch(0.40 0.17 264);
  --color-blue-600: oklch(0.30 0.17 264);

  --primary-color: var(--color-blue-500);
  --secondary-color: hsl(240, 48%, 55%);

  --primary-background-color: light-dark(var(--color-neutral-50), var(--color-neutral-700));
  --secondary-background-color: light-dark(var(--color-neutral-800), var(--color-neutral-800));

  --main-background-color: var(--primary-background-color);
  --nav-background-color: var(--secondary-background-color);
  --footer-background-color: var(--secondary-background-color);
}
```


CSS Functions

```
color-scheme: light dark;  
  
color: light-dark(#333b3c, #efefec);  
  
width: max(20vw, 400px);  
height: calc(10px + 100px);  
  
filter: brightness(1);  
backdrop-filter: blur(2px);  
  
color-mix(in lch longer hue, hsl(200deg 50% 80%), coral)
```

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Values_and_Units/CSS_Value_Functions
<https://developer.mozilla.org/en-US/docs/Web/CSS/filter-function>
https://developer.mozilla.org/en-US/docs/Web/CSS/color_value/color-mix

Webseiten interaktiv machen mit JavaScript

vgl.: Douglas Crockford (2018): How JavaScript Works, virgule solidus

How Class Free Works

- Klassen sind syntaktischer Zucker, d.h. sie bieten keine Funktionalität, die nicht mit anderen Mitteln erreicht werden kann.
- Sie verhalten sich anders als Klassen in C++, Java oder C#. Das kann verwirrend sein.

"Composition over Inheritance"

- Vererbung ist weniger zentral als manche Sprachen oder Kurse vermitteln.
- Vererbung bringt auch einige Probleme mit sich, da die Klassen sehr eng gekoppelt sind und nicht explizit klar ist, welche Methoden aufgerufen werden.
- Komposition ist sehr leistungsfähig.

Closures

Verschachtelte Funktionen können auf Variablen aus den äusseren Funktionen zugreifen. Auch nach deren Ausführung.

```
function init() {  
    const name = "Mozilla"; // name is a local variable created by init  
  
    function displayName() {  
        // displayName() is the inner function, that forms the closure  
        console.log(name); // use variable declared in the parent function  
    }  
}
```

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures>

Folgende Struktur wird empfohlen:

```
function counter_constructor() {  
  // private property  
  let counter = 0;  
  
  // composition  
  const reuse = other_constructor();  
  
  function up() {  
    counter -= 1;  
    return counter;  
  }  
  
  function down() {  
    counter += 1;  
    return counter;  
  }  
  
  // freeze to make the object immutable  
  return Object.freeze({  
    // make functions up and down public  
    up,  
    down,  
    // expose goodness property from another object  
    goodness: reuse.goodness  
  })  
}
```

Asynchronität

- JavaScript wurde primär für User-Interaktionen entwickelt.
- Asynchronität ist deshalb ein zentrales Sprachfeature.
- Es gibt verschiedene Möglichkeiten für asynchronen Code:
 - Callbacks
 - Promise
 - `async` / `await`

`async` / `await` ist verwirrend, weil damit Code produziert wird, der synchron aussieht, aber asynchron funktioniert.

Callback-Funktionen

- Callback-Funktionen werden als Parameter einer Funktion übergeben und von dieser aufgerufen.
- Die sogenannte "Callback-Hell", gemeint ist die Verschachtelung von Callbacks in Callbacks, sollte vermieden werden.

```
function foo(callback) {  
    // some functionality  
  
    callback(value);  
}  
  
foo((value) => {  
    // runs after "some functionality"  
})
```

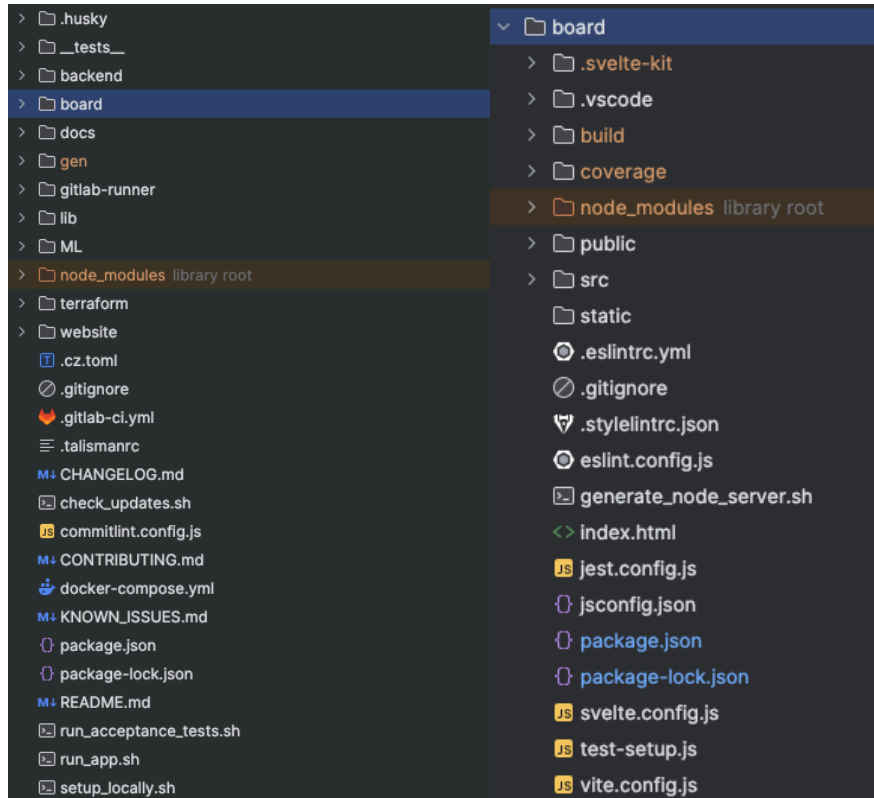

Promise

Promises können klarer sein als Callbacks, sind aber auch weniger explizit und potenziell verwirrend.

```
const p1 = new Promise((resolve, reject) => {  
  // some functionality  
  
  resolve("Success!");  
});  
p1.then((value) => {  
  // runs after "some functionality"  
})  
);
```

Single-Page-Applikationen implementieren

Typische Dokumentstruktur



- **Docs:** README.md, CONTRIBUTING.md, LICENCE.md, etc
- **Configs:** .gitignore, .talismanrc, commitlint.config.js, eslint.config.js, .dockerignore, jest.config.js, jsconfig.json,
- **Build & Deployment:** .gitlab-ci.yml, Dockerfile, docker-compose.yml, package.json, package-lock.json
- **Render & build outputs:** /dist, /build, /coverage, /node_modules
- **Source Code:** /src

Local First

1. No spinners: your work at your fingertips
 2. Your work is not trapped on one device
 3. The network is optional
 4. Seamless collaboration with your colleagues
 5. The Long Now
 6. Security and privacy by default
 7. You retain ultimate ownership and control
- <https://www.inkandswitch.com/local-first/>
 - <https://localfirstweb.dev/>

Webservices implementieren

- Level 0: The Swamp of POX
- Level 1: Resources
- Level 2: HTTP verbs

URI	HTTP verb	Operation
/destinations	GET	retrieve destinations
/reservations	GET	get reservations according to certain criteria
/reservations	POST	add/cancel reservations
/rooms	POST	request room extras
/rooms	DELETE	remove room extras

- Level 3: Hypermedia controls

```
{  
  customerId: "1",  
  reservations: [  
    {
```

Die Performance von Webanwendungen optimieren

- Nachhaltige Software wird immer wichtiger
- [Wirth's law](#):

"The hope is that the progress in hardware will cure all software ills. However, a critical observer may observe that software manages to outgrow hardware in size and sluggishness."

- Wettbewerbsvorteil
- <https://www.blauer-engel.de/de/produktwelt/software>

Energy, Time, Memory Comparision

Table 4: Normalized global results for Energy, Time, and Memory

Total					
	Energy (J)		Time (ms)		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84